Storage as a Functor

Definition (Schema-as-category design lens)

A *schema category* has one object per table and one morphism per foreign key. A *schema-conforming instance* is a functor from the schema category into $\mathbf{Set}$, sending each table object to the set of its rows and each foreign-key morphism to the corresponding function between row-sets. This template uses the term as a design lens (below), not as a mechanically checked property.

`src/template_formal/storage/schema.py` documents the agent-local SQLite schema as an instance of Fong and Spivak's functorial view of databases (Fong and Spivak (2018); see also the Ologs framework of Spivak and Kent (2012)) per the definition above. `Column` and `TableSchema` — the typed dataclasses this module defines — play the role of that schema category's objects: a `Column` is a typed field of a table, and a `TableSchema` is a table's full

column list plus the SQL DDL it compiles to, generated programmatically rather than hand-written at each call site (ISC-9).

What this framing is, and is not I

This is stated here exactly as it is stated in the source docstring: a *design lens*, not a load-bearing mathematical claim. Nothing in `storage/schema.py`, `storage/db.py`, or their tests checks the actual category-theoretic functoriality laws — identity-preservation and composition-preservation — against this schema. A forker who wanted that stronger guarantee would need a genuine categorical-database library (or a proof assistant encoding) checking those laws against the schema's foreign keys; this template does not attempt that, and does not claim to. The value the framing does deliver, honestly: it gives the typed query builder (`storage/db.py`'s `QueryBuilder[RowT]`, generic over the row type) a principled vocabulary for what a "row set" and a "schema" are, and it makes explicit that a schema *is* structured data with morphisms between

What this framing is, and is not I

its parts — a genuinely useful design lens for reasoning about foreign-key integrity — without over-promising a machine-checked proof the codebase does not deliver.

Affine-discipline transactions over that instance I

Every write to an agent's schema-instance goes through exactly one `TransactionHandle` per transaction (`storage/transaction.py`), consumed at most once (sec. `sec:honesty-line`). A real on-disk SQLite file (`tmp_path`-backed, never `:memory:`-only, per ISC-66) is used in every test that makes a durability claim, including the rollback test that asserts — via a real `SELECT`, not an assumption — that a rolled-back transaction leaves the database in its exact pre-transaction state (ISC-14). The typed query builder never raises for an *expected* failure mode (a missing row, a constraint violation); those come back as `Result.Err(StorageError(...))` (ISC-10), keeping expected failure on the ADT side of the line drawn in sec. `sec:honesty-line` and reserving raised exceptions for genuine

Affine-discipline transactions over that instance I

programmer errors such as affine-handle reuse.

Per-agent isolation I

Per the ISA's decentralization framing (`Out of Scope`: "independent local DB … per agent, no shared global state"), each `Agent` (`src/template_formal/agent/agent.py`) opens its own `Database` at construction time and never exposes it — there is no public attribute or method on `Agent` that returns a `Path`, `sqlite3.Connection`, or `Database`, and none that accepts one either. `tests/agent/test_agent_isolation.py` confirms this structurally (ISC-30): a second agent's storage file path is unreachable through the first agent's public API, not merely unreached in the tests that happen to exist.